



Artificial Intelligence in Medicine: Foundations and Clinical Applications

Instructor



Sara El-Sayed El-Metwally

Associate Professor of Computer Science,
Computer Science Department,
Faculty of Computers and Information,
Mansoura University



<https://sarael-metwally.github.io/SaraEl-MetwallyLab/>



sarah_almetwally4@mans.edu.eg

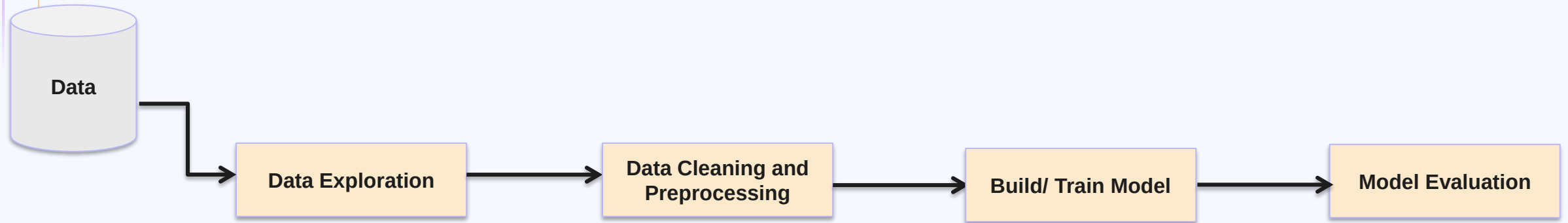


<https://www.facebook.com/sara.almetwally>

AI Workflow

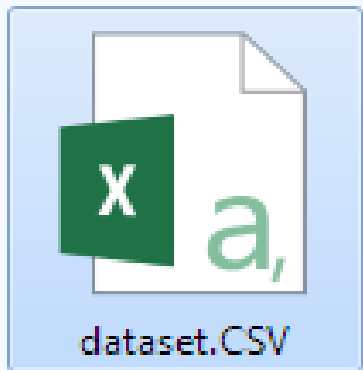


AI Workflow

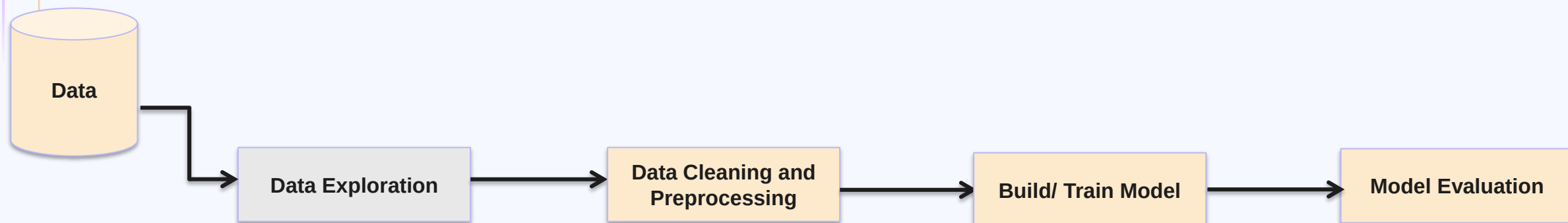


Dataset Overview

- **Observations:** 768 patient records (500 negative, 258 positive)
- **Target Variable:** Outcome (1 indicates diabetes, 0 indicates no diabetes)
- **Predictor Variables:** There are 8 features for each patient:
 1. **Pregnancies:** Number of times pregnant
 2. **Glucose:** Plasma glucose concentration from an oral glucose tolerance test
 3. **BloodPressure:** Diastolic blood pressure in mm Hg
 4. **SkinThickness:** Triceps skin fold thickness in mm
 5. **Insulin:** 2-Hour serum insulin in mu U/ml
 6. **BMI:** Body mass index (weight in kg / height in m²)
 7. **DiabetesPedigreeFunction:** Function indicating the likelihood of diabetes based on family history
 8. **Age:** Age in years

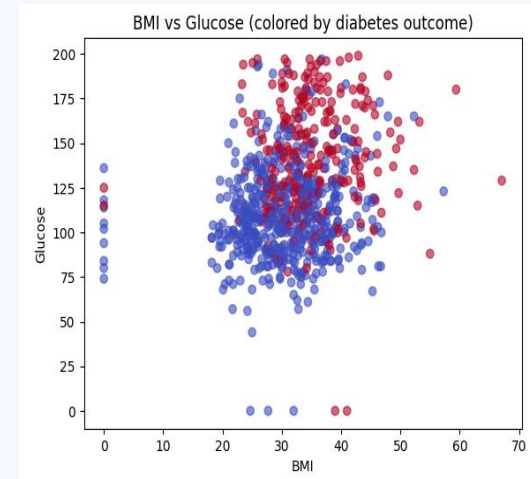
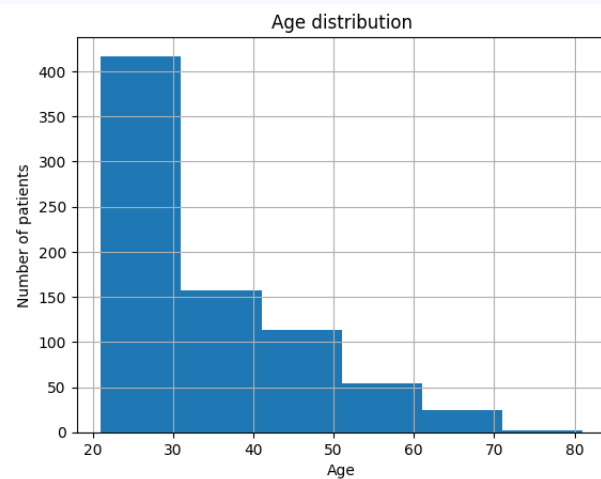


AI Workflow

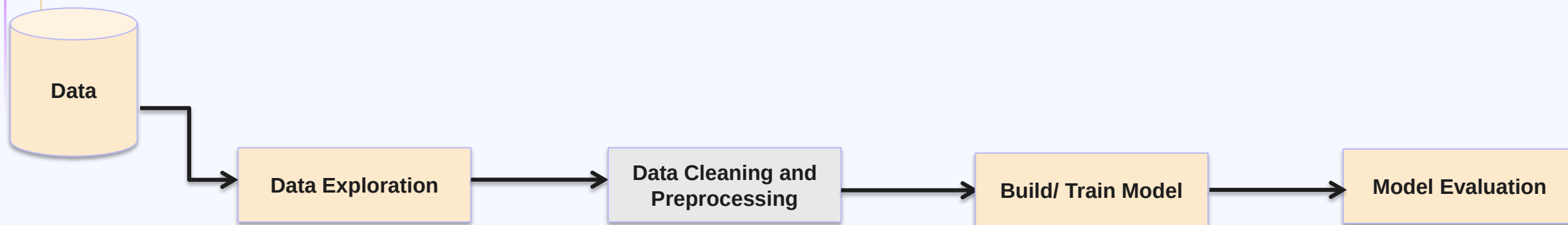


```
data.describe() # mean, min, max, etc. for each column
```

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

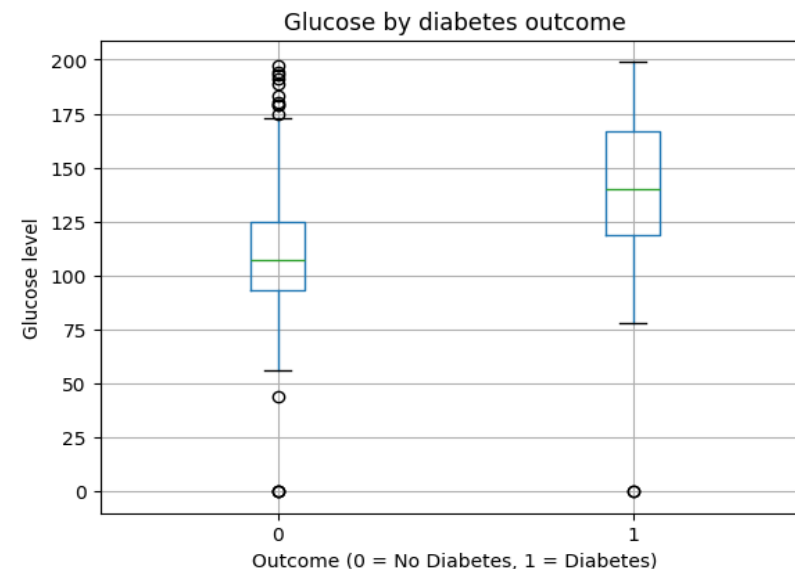


AI Workflow



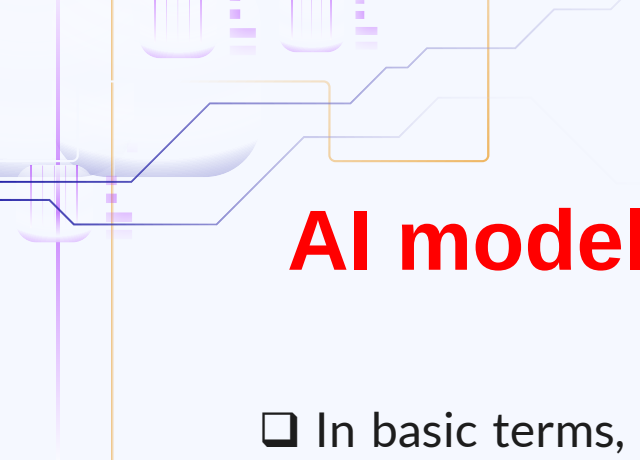
```
data.describe() # mean, min, max, etc. for each column
```

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000



AI Workflow



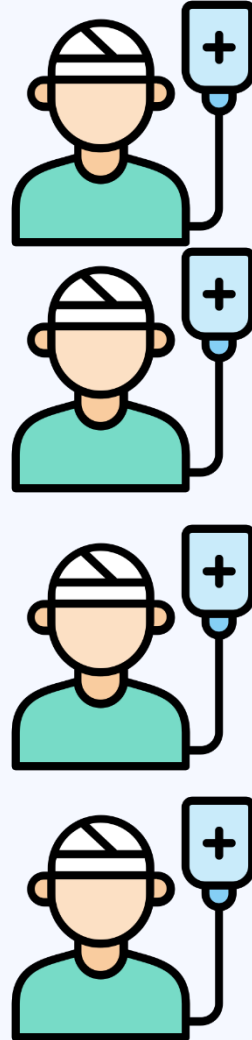


AI model

- ❑ In basic terms, ML is the process of training a piece of software, called a **model**, to make useful predictions from data.
- ❑ For example, suppose we wanted to create an app to predict rainfall.
 - ❑ Using an ML approach, we would give an ML model enormous amounts of patient health data (such as blood pressure, cholesterol, age, and ECG results) until the ML model eventually learned the mathematical relationship between clinical metrics and heart disease risk (**Training**).
 - ❑ We would then give the model a new patient's current health data, and it would predict their risk of heart disease (**Testing**).

AI Model

Training



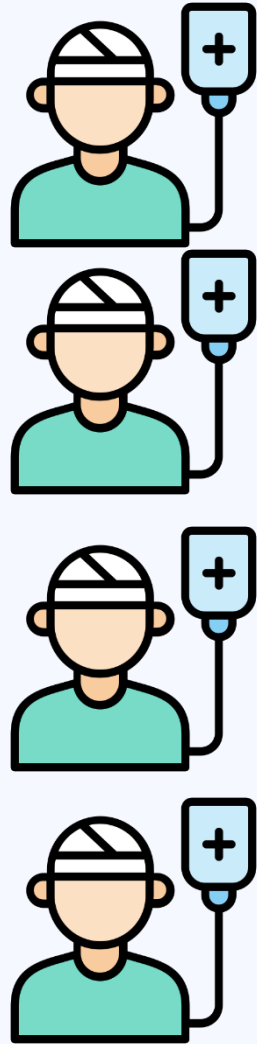
Training Patients on a Specific Disease

Testing

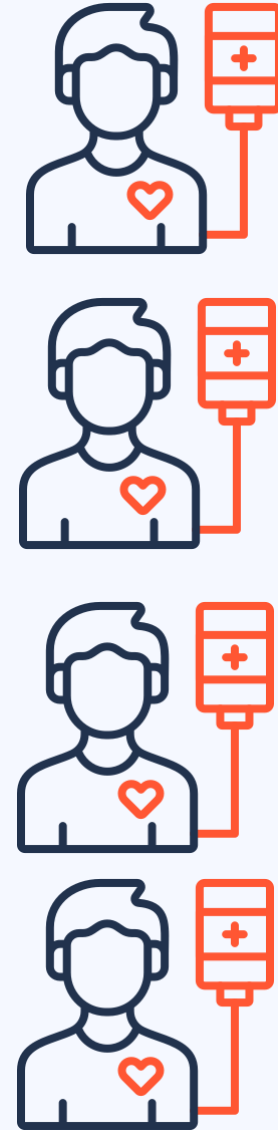
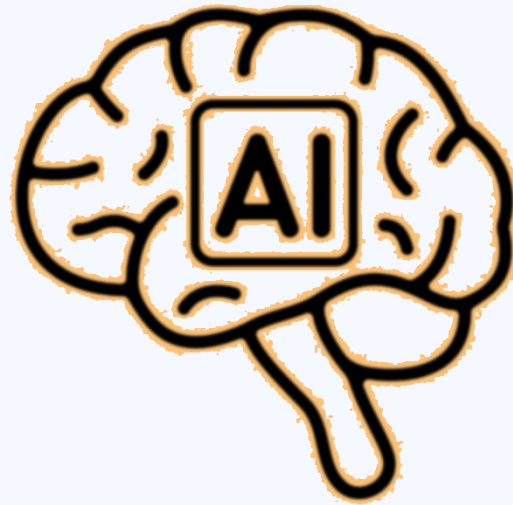


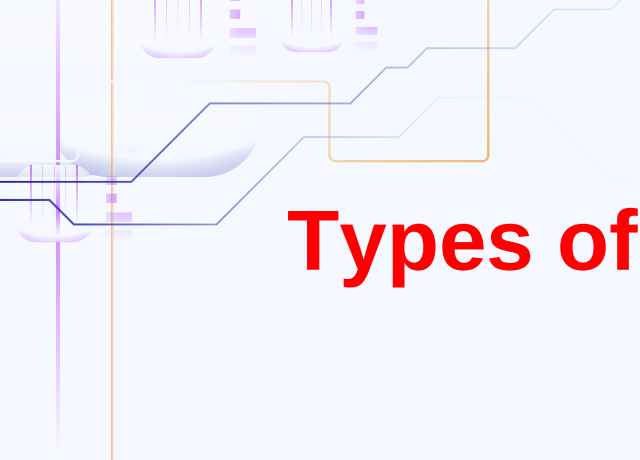
Real-life Patients on your Clinic

Training Data



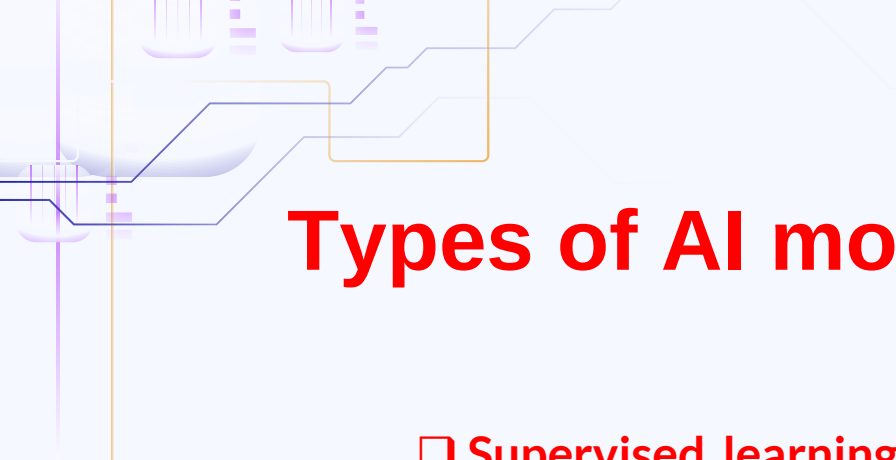
Testing Data





Types of AI models

- ☐ Supervised learning
- ☐ Unsupervised learning
- ☐ Reinforcement learning
- ☐ Generative AI



Types of AI models

❑ **Supervised learning:** can make predictions after seeing lots of data with the correct answers and then discovering the connections between the elements in the data that produce the correct answers. Two of the most common use cases for supervised learning are **regression** and **classification**.

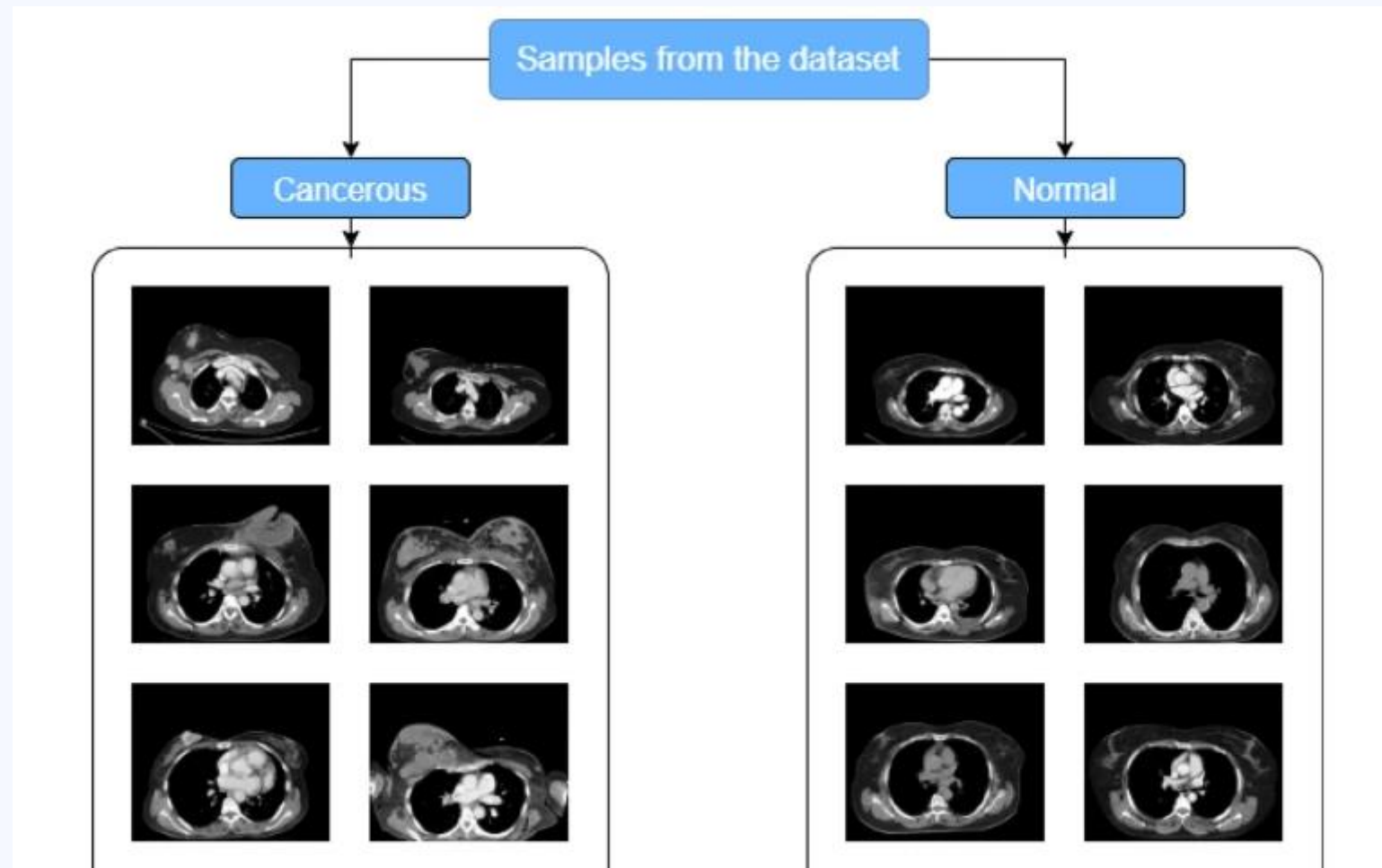
❑ **A regression model** predicts **a numeric value**. For example a medical model that predicts:

- a patient's blood glucose level, in mg/dL
- a patient's systolic blood pressure, in mmHg
- predicts a patient's length of hospital stay, in days
- predicts the size of a tumor, in millimeters

Types of AI models

- ❑ Supervised learning
- ❑ Unsupervised learning
- ❑ Reinforcement learning
- ❑ Generative AI

Breast Cancer CT-Scan Image Classification



Class/Label= Cancerous

Class/Label= Normal

Types of AI models

FIGURE 1:

SIMPLIFIED INDIVIDUAL PATIENT GLUCOSE PREDICTIONS

Track A:
Adult Female,
Active Life



INPUT DATA:



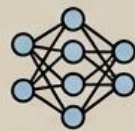
Age: 45



BMI: 24



Meal: 2 hrs ago
(small snack)



ML PREDICTION MODEL
(SINGLE TIME-POINT
CALCULATION)



**Predicted
Glucose:**
110 mg/dL
(Single numeric value
for this moment)

Track B:
Senior Male,
Sedentary Life



INPUT DATA:



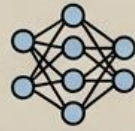
Age: 72



BMI: 31



Meal: 4 hrs ago
(full meal)



ML PREDICTION MODEL
(SINGLE TIME-POINT
CALCULATION)

**Predicted
Glucose:**
165 mg/dL
(Single numeric value
for this moment)

Track C:
Child Male,
High Activity



INPUT DATA:



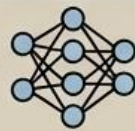
Age: 10



BMI: 18



Meal: 1 hr ago
(lunch)

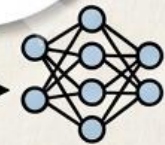


ML PREDICTION MODEL
(SINGLE TIME-POINT
CALCULATION)



**Predicted
Glucose:**
95 mg/dL
(Single numeric value
for this moment)

EXAMPLE: An ML model is applied to current data for three different patients at a single, specific time-point. The model takes their individual input features and calculates a unique, precise numeric prediction for their glucose level.





Classification AI models

❑ **Classification models** predict the likelihood that something belongs to a category.

❑ Classification models output a value that states whether or not something belongs to a particular category. For example, classification models are used to predict if a patient has cancer or not.

❑ Classification models are divided into two groups: **binary classification** and **multiclass classification**.

❑ **Binary classification** models output a value from a class that contains only two values, for example, a model that outputs either cancer or no cancer.

❑ **Multiclass classification** models output a value from a class that contains more than two values, for example, a model that can output either breast, colon, oral cancers.

Classification AI models

❑ **Example:** using gene expression data, you can predict different types of cancer.

	gene_1	gene_2	gene_3	gene_4	gene_7999	gene_8000	Cancer_Type
0	0.0	0.000000	0.0	2.088413	6.928584	2.088413	KIRC
1	0.0	0.000000	0.0	3.205955	5.039231	2.448002	KIRC
2	0.0	0.000000	0.0	4.746646	3.119738	4.433988	BRCA
3	0.0	0.000000	0.0	1.173191	2.863046	3.380342	KIRC
4	0.0	0.000000	0.0	1.366532	6.544487	0.839395	COAD
...	...	...	...	...	...	...	...
796	0.0	2.933384	0.0	1.683921	6.396848	1.252839	BRCA
797	0.0	0.000000	0.0	2.772625	5.386625	1.697640	COAD
798	0.0	0.408277	0.0	1.854754	5.507417	1.398131	BRCA
799	0.0	0.639232	0.0	3.226971	5.282158	0.639232	BRCA
800	0.0	0.000000	0.0	2.336912	7.550000	0.926379	COAD

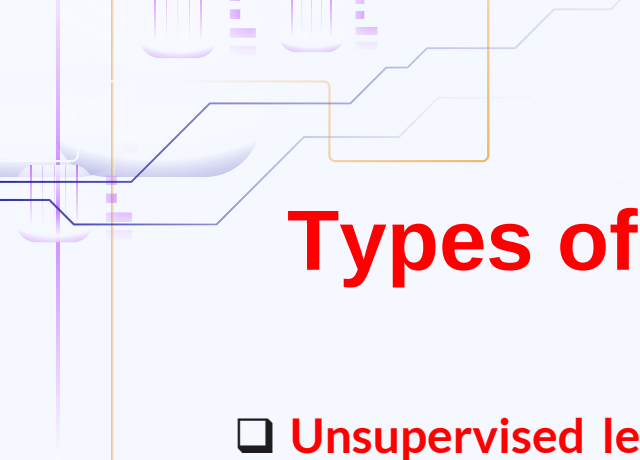
Kidney renal clear cell carcinoma (**KIRC**)

Breast invasive carcinoma (**BRCA**)

Colon adenocarcinoma (**COAD**)

Lung adenocarcinoma (**LUAD**)

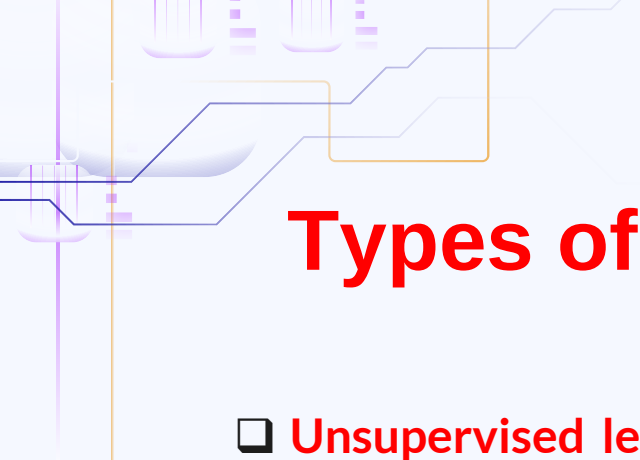
Prostate adenocarcinoma (**PRAD**)



Types of AI models

❑ **Unsupervised learning models** make predictions by being given data that does not contain any correct answers. An unsupervised learning model's goal is to identify meaningful patterns among the data. In other words, the model has no hints on how to categorize each piece of data, but instead it must infer its own rules.

❑ **For example**, an unsupervised model might cluster a patient dataset based on vital signs and lab results, revealing segmentations that define distinct risk profiles. You might then attempt to name those clusters (such as "**Healthy**," "**Pre-Diabetic**," or "**High Risk**") based on your understanding of the dataset.

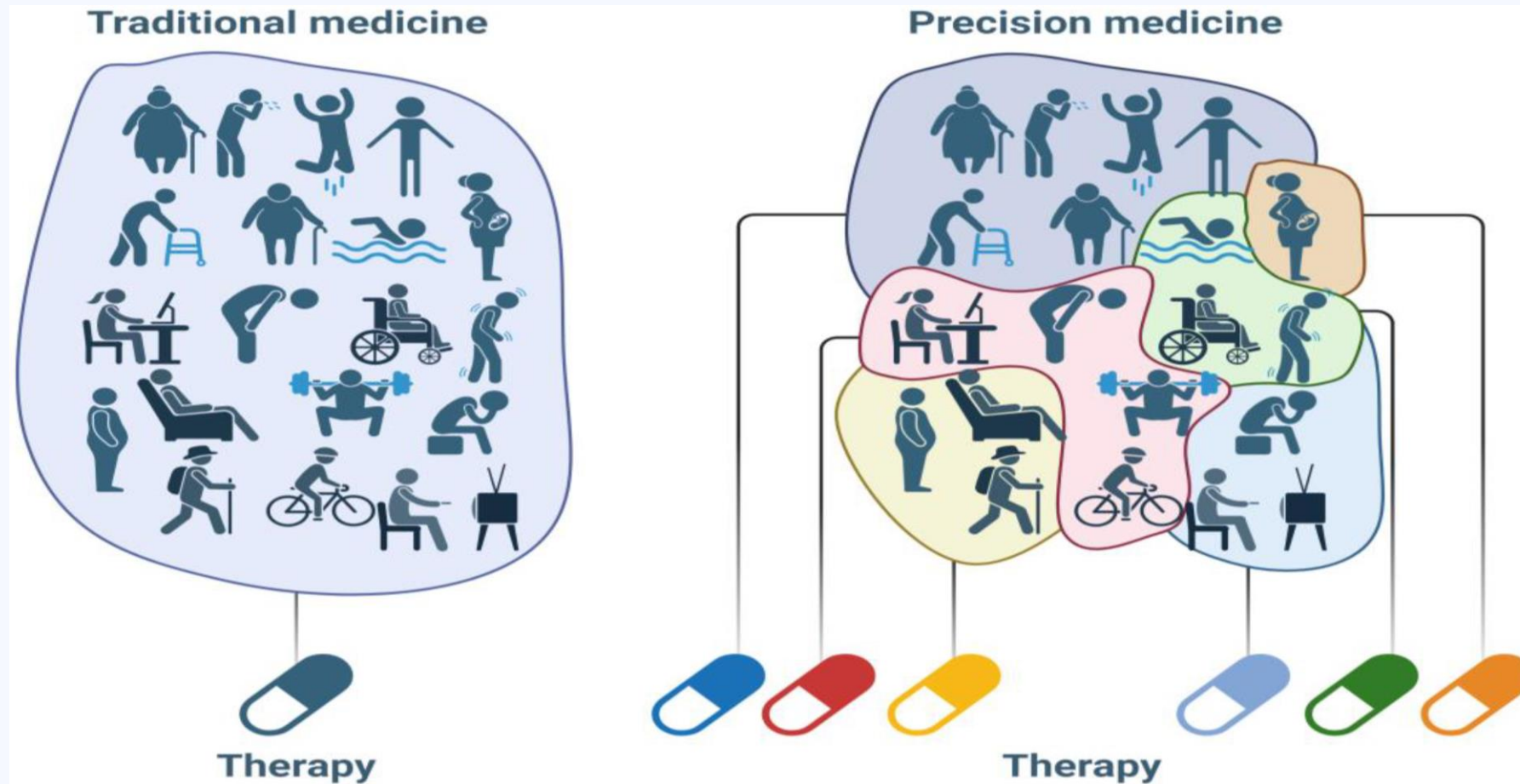


Types of AI models

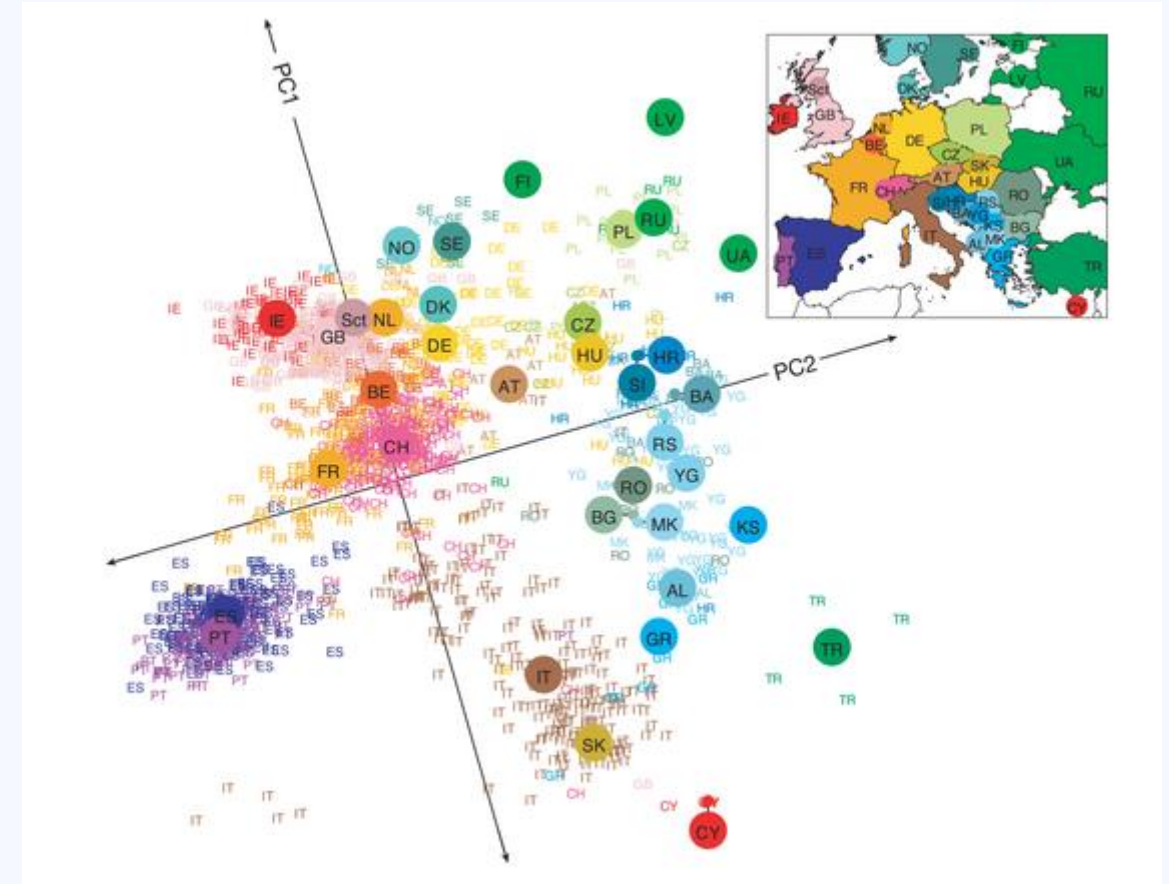
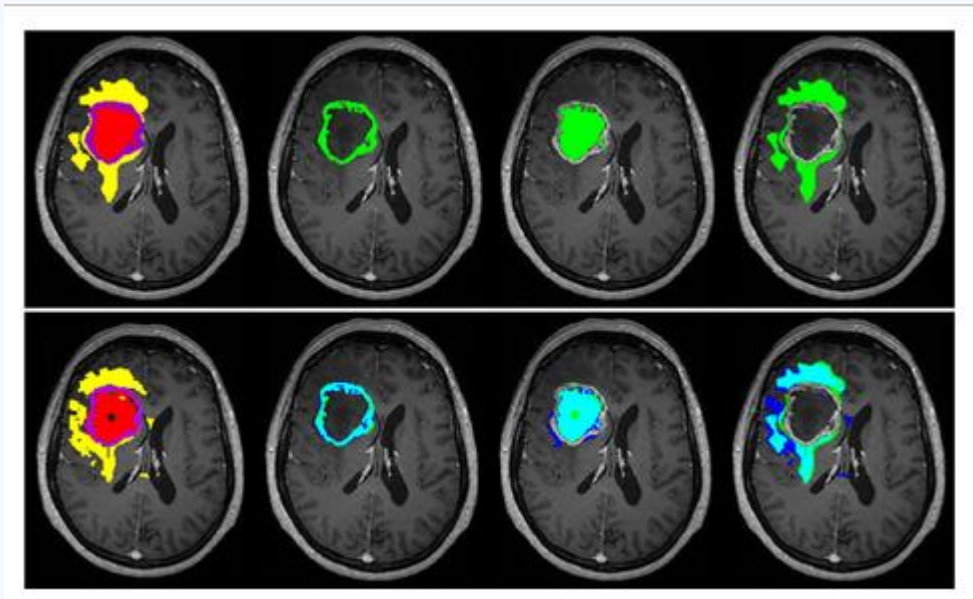
❑ **Unsupervised learning models** make predictions by being given data that does not contain any correct answers. An unsupervised learning model's goal is to identify meaningful patterns among the data. In other words, the model has no hints on how to categorize each piece of data, but instead it must infer its own rules.

❑ **For example**, an unsupervised model might cluster a patient dataset based on vital signs and lab results, revealing segmentations that define distinct risk profiles. You might then attempt to name those clusters (such as "**Healthy**," "**Pre-Diabetic**," or "**High Risk**") based on your understanding of the dataset.

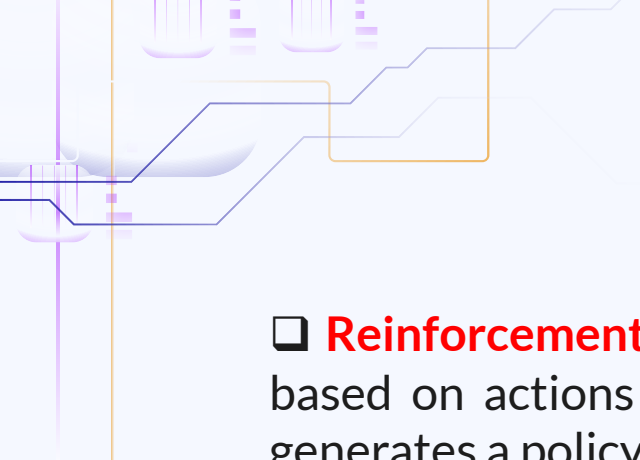
Types of AI models



Types of AI models



Genetic data for individuals from different populations in Europe



Types of AI models

❑ **Reinforcement learning** models make predictions by getting rewards or penalties based on actions performed within an environment. A reinforcement learning system generates a policy that defines the best strategy for getting the most rewards.

❑ For example, a reinforcement learning model can learn to optimize insulin dosing for diabetic patients.

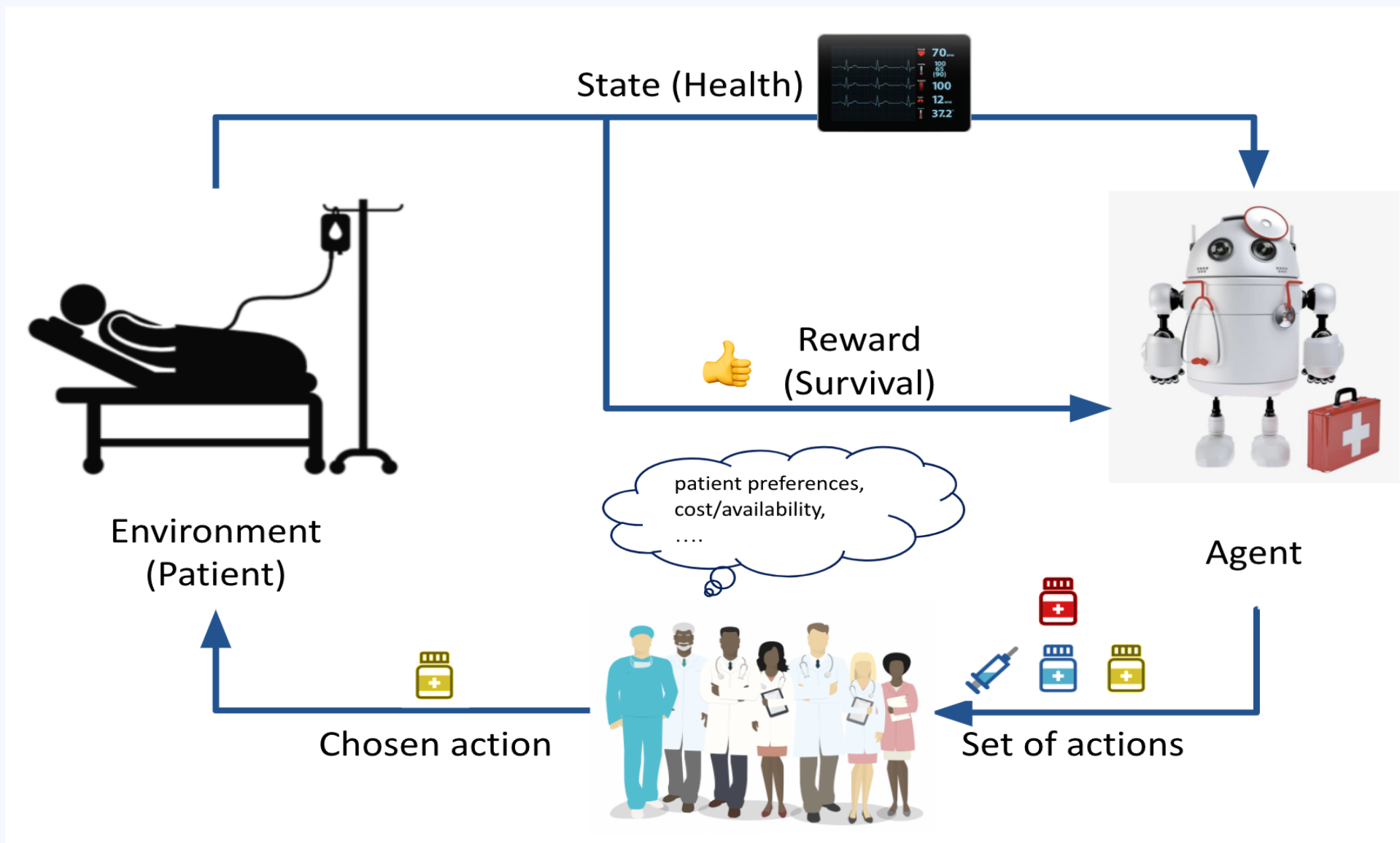
- The AI agent observes the patient's current blood glucose level and recent meal data (**State**).
- It decides on an optimal insulin dose to deliver (**Action**).
- It receives positive feedback if blood glucose normalizes, or a penalty if sugar spikes too high or drops too low (**Reward**), continually improving its treatment policy through trial and error over time.

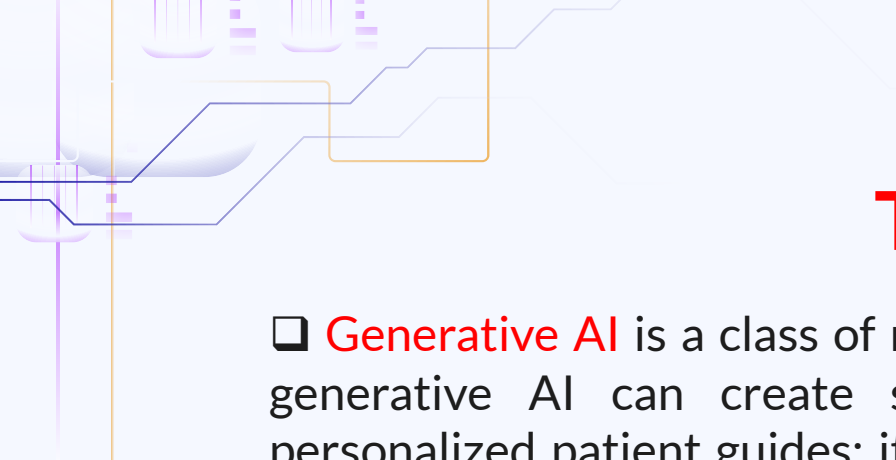
How Reinforcement Learning Maps to Medicine

❑ Unlike Supervised Learning (which learns from labeled historical data) or Unsupervised Learning (which finds hidden patterns), Reinforcement Learning learns by making decisions and receiving feedback (rewards/penalties).

RL Component	Medical Concept Example (Sepsis / ICU Treatment)
Agent	The AI clinical decision support system
Environment	The patient's body in the Intensive Care Unit (ICU)
State (S)	Patient's current vital signs (heart rate, blood pressure, oxygen, lab results)
Action (A)	Administering a specific dosage of medication or fluid adjustment
Reward (R)	Positive: Stabilization of organ function / patient recovery Negative (Penalty): Sudden drop in blood pressure or adverse side effects

How Reinforcement Learning Maps to Medicine





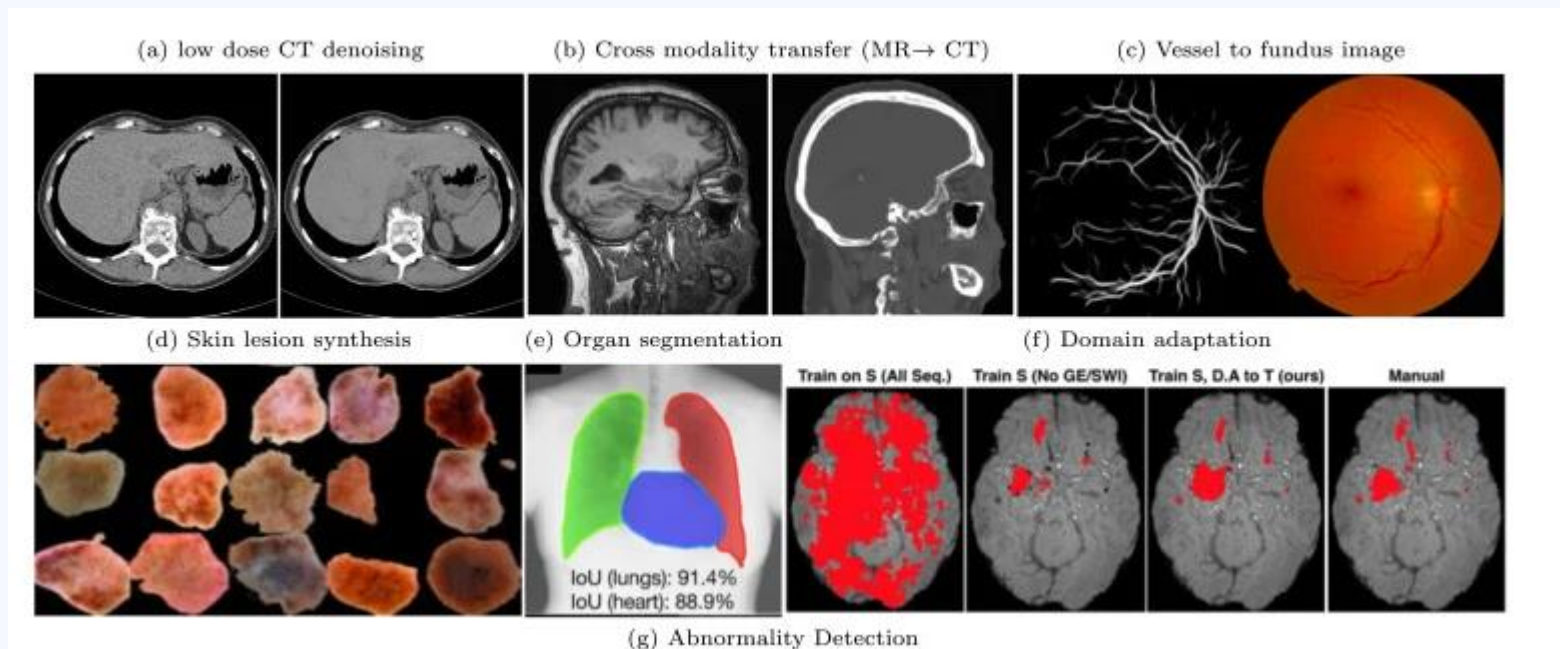
Types of AI models

❑ **Generative AI** is a class of models that creates content from user input. For example, generative AI can create synthetic medical images, novel drug molecules, and personalized patient guides; it can summarize clinical notes, explain treatment plans, or enhance radiology scans.

❑ **Generative AI** can take a variety of inputs and create a variety of outputs, like text, images, audio, and video. It can also take and create combinations of these. For example, a model can take an X-ray image as input and create an annotated scan and diagnostic report as output, or take a CT scan and physician notes as input and create a 3D surgical simulation video as output.

Types of AI models

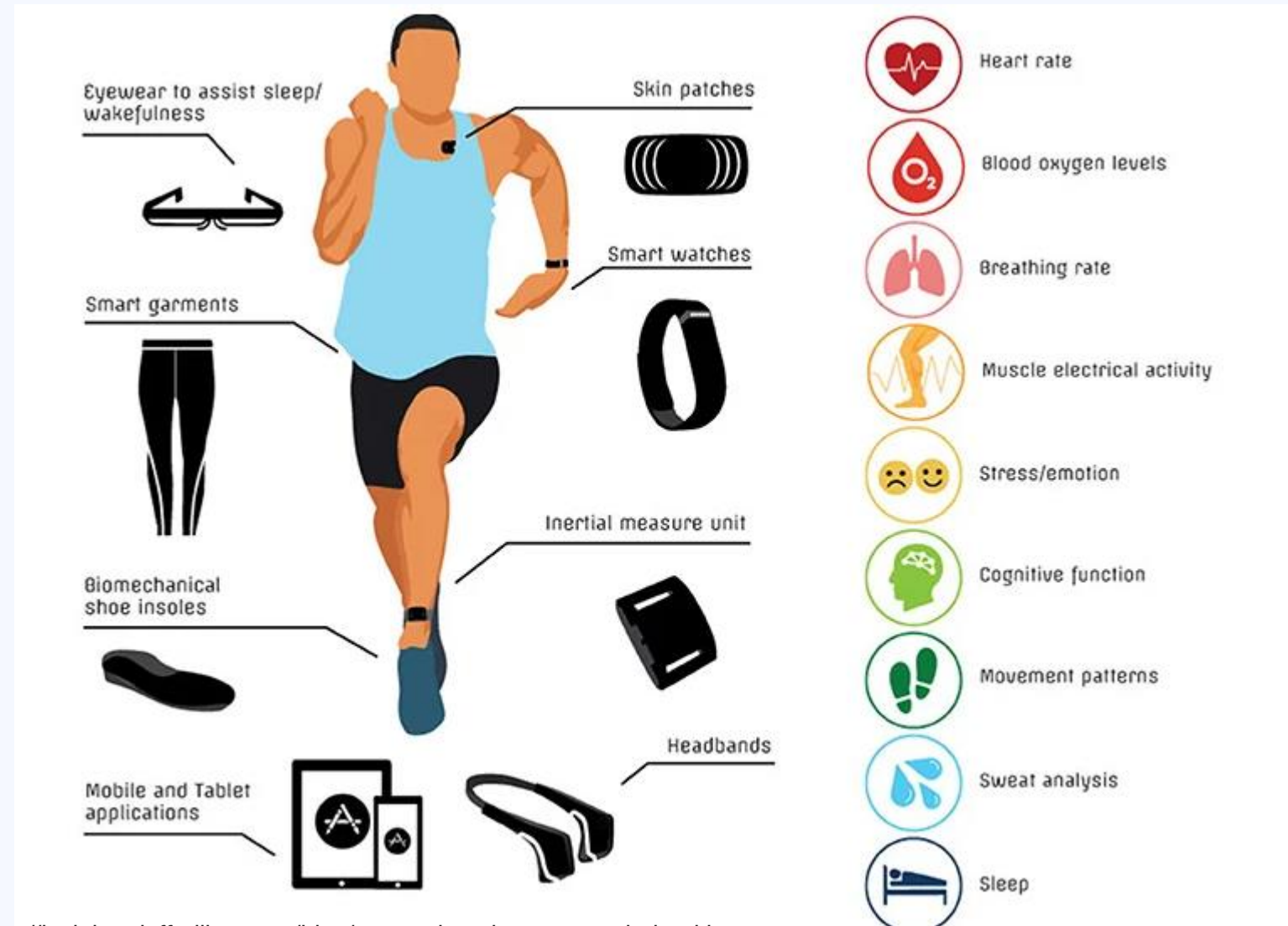
Generative AI can enhance image clarity by reducing background noises



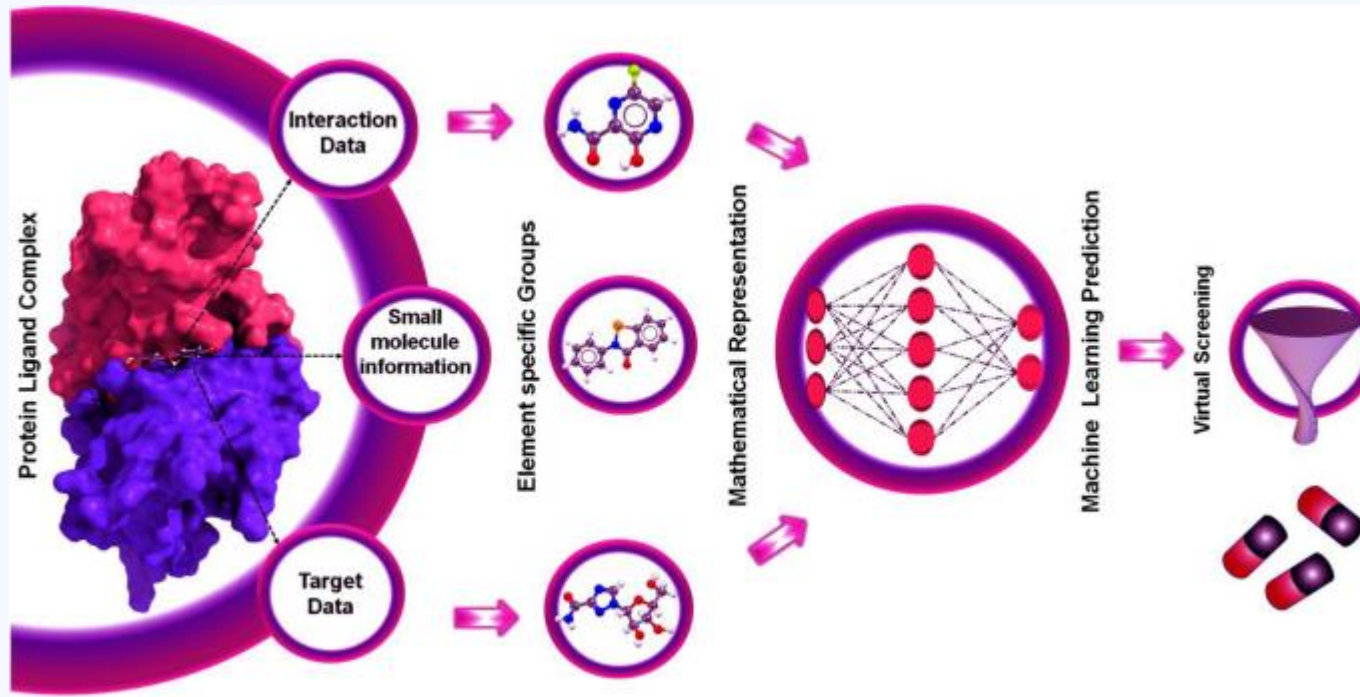
<https://insights.daffodilsw.com/blog/generative-ai-use-cases-in-healthcare>

Types of AI models

Companies developing medical devices are utilizing **GenAI** to design smart tools that assist in patient care

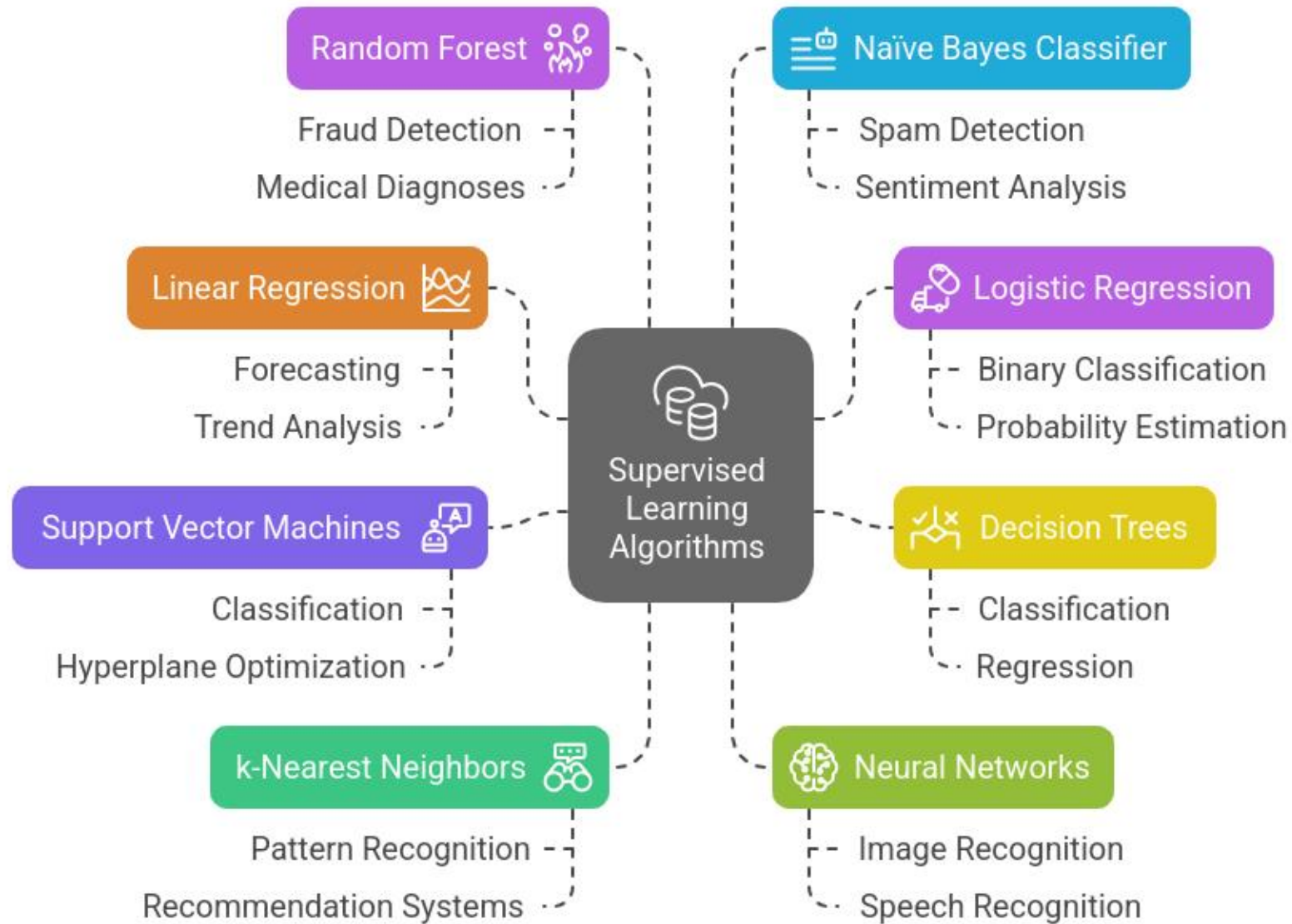


Types of AI models

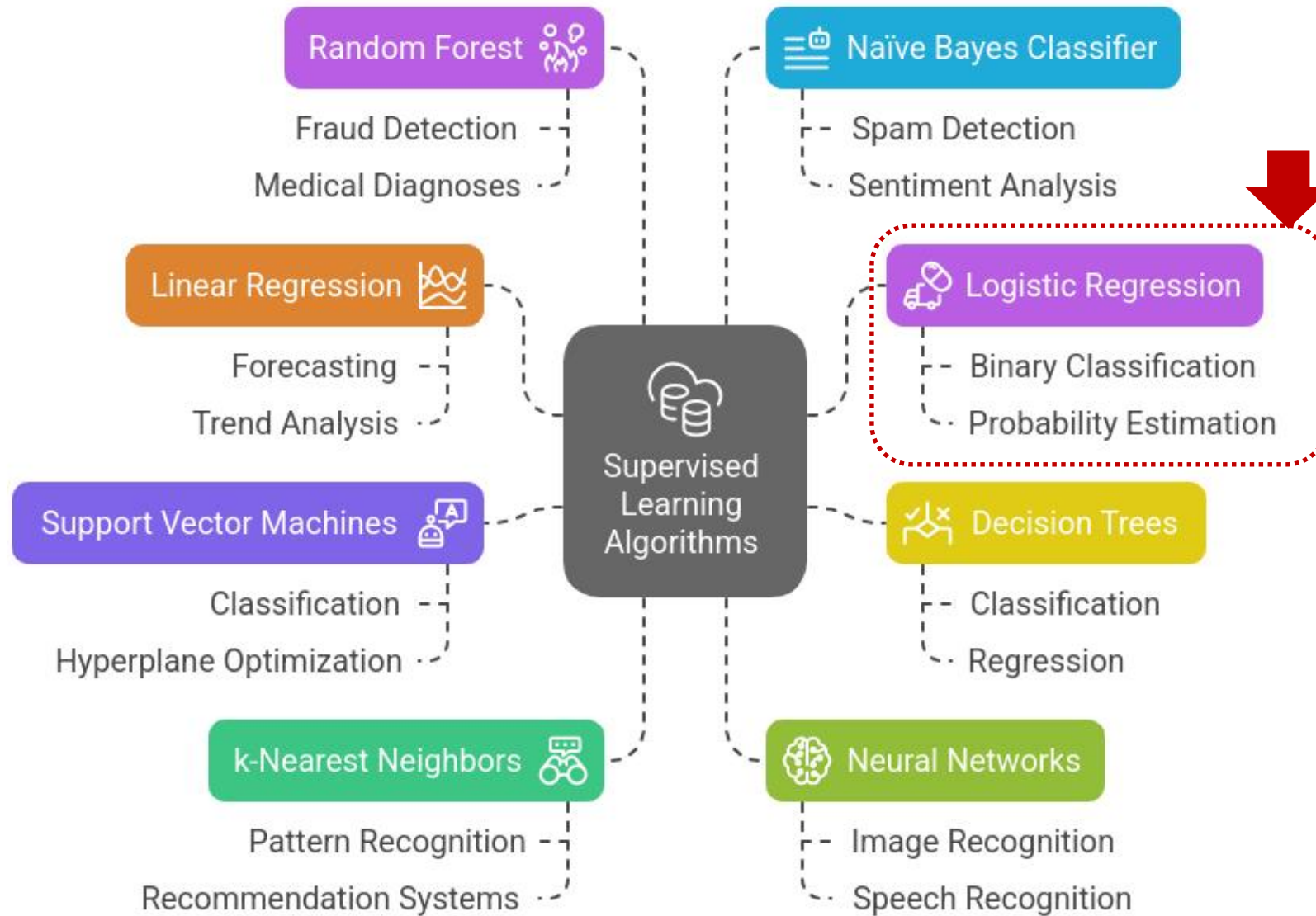


Generative AI can create virtual molecules. By leveraging GenAI, pharmaceutical scientists can develop these virtual compounds and evaluate them using computer simulations instead of conducting physical experiments.

Supervised Learning Algorithms and Their Applications



Supervised Learning Algorithms and Their Applications



Case Study: Diabetes Dataset

Dataset Overview

- **Observations:** 768 patient records (500 negative, 258 positive)
- **Target Variable:** Outcome (1 indicates diabetes, 0 indicates no diabetes)

- **Predictor Variables:** There are 8 features for each patient:
 1. **Pregnancies:** Number of times pregnant
 2. **Glucose:** Plasma glucose concentration from an oral glucose tolerance test
 3. **BloodPressure:** Diastolic blood pressure in mm Hg
 4. **SkinThickness:** Triceps skin fold thickness in mm
 5. **Insulin:** 2-Hour serum insulin in mu U/ml
 6. **BMI:** Body mass index (weight in kg / height in m²)
 7. **DiabetesPedigreeFunction:** Function indicating the likelihood of diabetes based on family history
 8. **Age:** Age in years

Inputs

Case Study: Diabetes Dataset

outputs

Dataset Overview

- **Observations:** 768 patient records (500 negative, 258 positive)
- **Target Variable:** Outcome (1 indicates diabetes, 0 indicates no diabetes)
- **Predictor Variables:** There are 8 features for each patient:
 1. **Pregnancies:** Number of times pregnant
 2. **Glucose:** Plasma glucose concentration from an oral glucose tolerance test
 3. **BloodPressure:** Diastolic blood pressure in mm Hg
 4. **SkinThickness:** Triceps skin fold thickness in mm
 5. **Insulin:** 2-Hour serum insulin in mu U/ml
 6. **BMI:** Body mass index (weight in kg / height in m²)
 7. **DiabetesPedigreeFunction:** Function indicating the likelihood of diabetes based on family history
 8. **Age:** Age in years

Case Study: Diabetes Dataset

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Inputs = Features = X

Case Study: Diabetes Dataset

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Output = Correct Answer = y

Case Study: Diabetes Dataset

```
X = data.drop("outcome", axis=1)  
y = data["outcome"]
```

drop



	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

data

Case Study: Diabetes Dataset

```
X = data.drop("outcome", axis=1)  
y = data["outcome"]
```

axis =1
dropping a column

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

data

Case Study: Diabetes Dataset

```
X = data.drop("outcome", axis=1)  
y = data["outcome"]
```

axis = 0
dropping a row

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

data

Case Study: Diabetes Dataset

	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

data

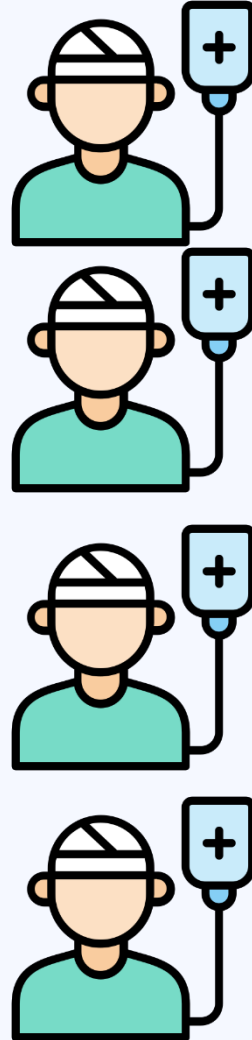
y

X

```
X = data.drop("outcome", axis=1)
y = data["outcome"]
```

AI Model

Training



Training Patients on a Specific Disease

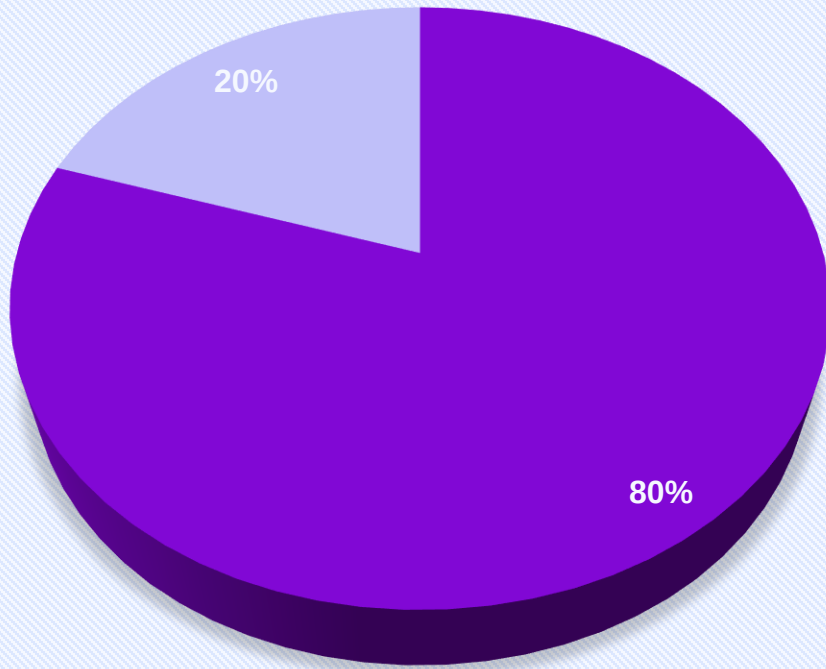
Testing



Real-life Patients on your Clinic

Case Study: Diabetes Dataset

Diabetes Data



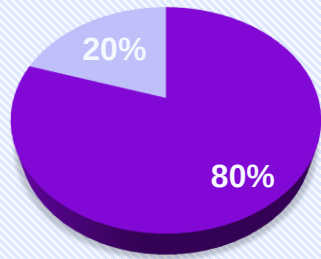
■ Training ■ Testing

Dataset Overview

- **Observations:** 768 patient records (500 negative, 258 positive)
- **Target Variable:** Outcome (1 indicates diabetes, 0 indicates no diabetes)
- **Predictor Variables:** There are 8 features for each patient:
 1. **Pregnancies:** Number of times pregnant
 2. **Glucose:** Plasma glucose concentration from an oral glucose tolerance test
 3. **BloodPressure:** Diastolic blood pressure in mm Hg
 4. **SkinThickness:** Triceps skin fold thickness in mm
 5. **Insulin:** 2-Hour serum insulin in μ U/ml
 6. **BMI:** Body mass index (weight in kg / height in m^2)
 7. **DiabetesPedigreeFunction:** Function indicating the likelihood of diabetes based on family history
 8. **Age:** Age in years

Case Study: Diabetes Dataset

Diabetes Data



■ Training ■ Testing

```
from sklearn.model_selection import train_test_split
# Stratify ensures both train and test sets maintain the original proportion of diabetes cases
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)
```

AI Model

Training



	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

X_train

y_train

Testing



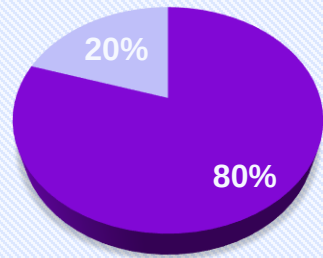
	pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age	outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

X_test

y_test

Case Study: Diabetes Dataset

Diabetes Data



■ Training ■ Testing

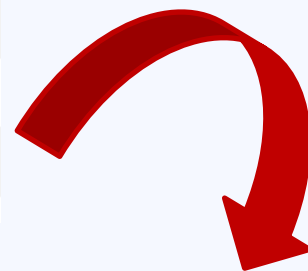
```
from sklearn.model_selection import train_test_split
# Stratify ensures both train and test sets maintain the original proportion of diabetes cases
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)
```

stratify=y: Ensures the train and test splits retain the identical proportion of class labels as the original dataset y . This prevents data imbalance issues during splitting.

Case Study: Diabetes Dataset

pregnancies	glucose	blood_pressure	skin_thickness	insulin	bmi	diabetes_pedigree	age
1	90.0	62.0	12.0	43.0	27.2	0.580	24
5	126.0	78.0	27.0	22.0	29.6	0.439	40
2	105.0	58.0	40.0	94.0	34.9	0.225	25
1	146.0	56.0	29.0	125.0	29.7	0.564	29
0	95.0	64.0	39.0	105.0	44.6	0.366	22

**Standardize features
(mean=0, std=1)**



X_train

	0	1	2	3	4	5	6	7
0	-0.851355	-1.056427	-0.826740	-1.918187	-1.203361	-0.769477	0.310794	-0.792169
1	0.356576	0.144399	0.477772	-0.229874	-1.470195	-0.417498	-0.116439	0.561034
2	-0.549372	-0.556083	-1.152868	1.233330	-0.555335	0.359790	-0.764862	-0.707594
3	-0.851355	0.811525	-1.315932	-0.004766	-0.161437	-0.402832	0.262314	-0.369293
4	-1.153338	-0.889646	-0.663676	1.120776	-0.415565	1.782373	-0.337630	-0.961320

X_train_scaled

Case Study: Diabetes Dataset

```
from sklearn.preprocessing import StandardScaler
# Standardize features (mean=0, std=1) so coefficients reflect feature importance
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- **scaler = StandardScaler()**, Initializes the scikit-learn StandardScaler object. This scaler standardizes features by centering them to a mean of 0 and scaling them to a standard deviation of 1.
- **X_train_scaled = scaler.fit_transform(X_train)**, **fit** calculates the mean (μ) and standard deviation (σ) of each feature strictly from the training dataset (X_{train}). **transform** uses those calculated metrics to scale the X_{train} data.

Case Study: Diabetes Dataset

```
from sklearn.preprocessing import StandardScaler
# Standardize features (mean=0, std=1) so coefficients reflect feature importance
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

How to Calculate It

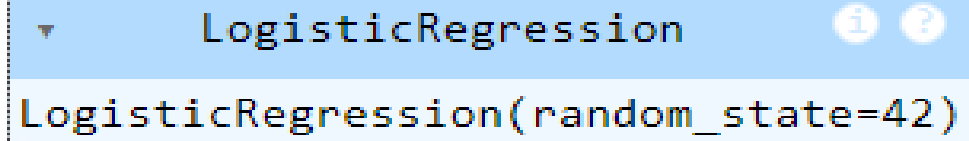
- **Formula:** $Z = \frac{X - \mu}{\sigma}$
- **Step 1:** Find the mean (μ) of the variable.
- **Step 2:** Find the standard deviation (σ) of the variable.
- **Step 3:** Subtract the mean from each data point.
- **Step 4:** Divide that difference by the standard deviation.

Why We Do It

- **Equal Scale:** Puts variables with different units or ranges onto the same scale.
- **Fair Weight:** Stops large numbers from dominating machine learning models or statistical tests.

Case Study: Diabetes Dataset

```
from sklearn.linear_model import LogisticRegression
# 4. TRAIN MODEL
# =====
model = LogisticRegression(random_state=42)
model.fit(X_train_scaled, y_train)
```



LogisticRegression

LogisticRegression(random_state=42)

- **LogisticRegression()** : Imports the classification algorithm used to predict binary classification.
- **fit()** : The command that triggers the actual training/learning process.

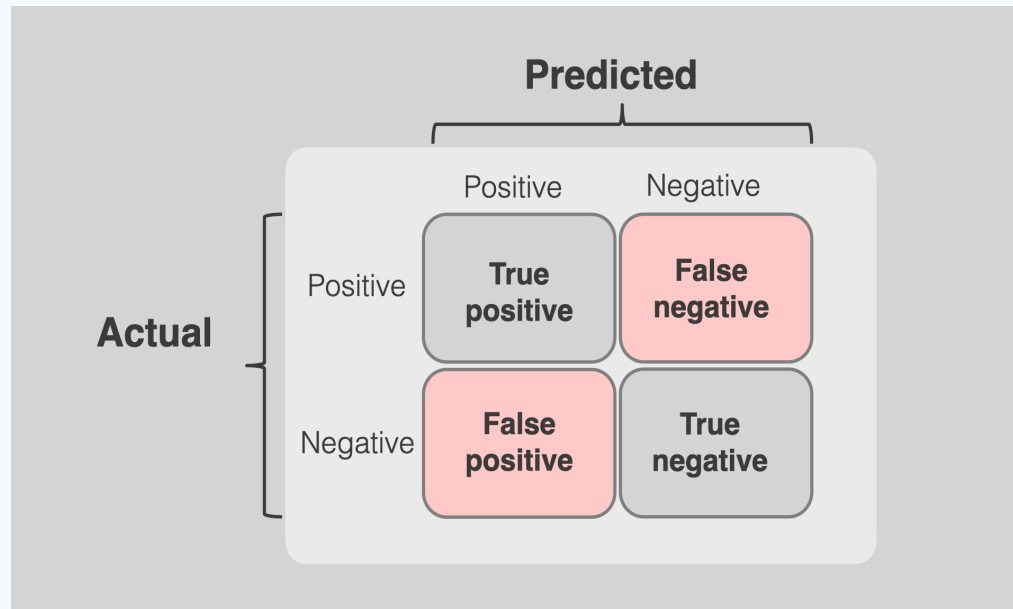
Case Study: Diabetes Dataset

```
# 5. CLINICAL EVALUATION
# =====
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[: , 1] # Risk probabilities
```

- **`model.predict(X_test_scaled)`** :
 - Outputs final discrete class choices (e.g., 0 or 1).
- **`model.predict_proba(X_test_scaled)`** :
 - Outputs a 2D array of class probability estimates for every sample.
 - Column 0 holds the probability of the negative class ($P(y=0)$), and column 1 holds the probability of the positive class ($P(y=1)$)

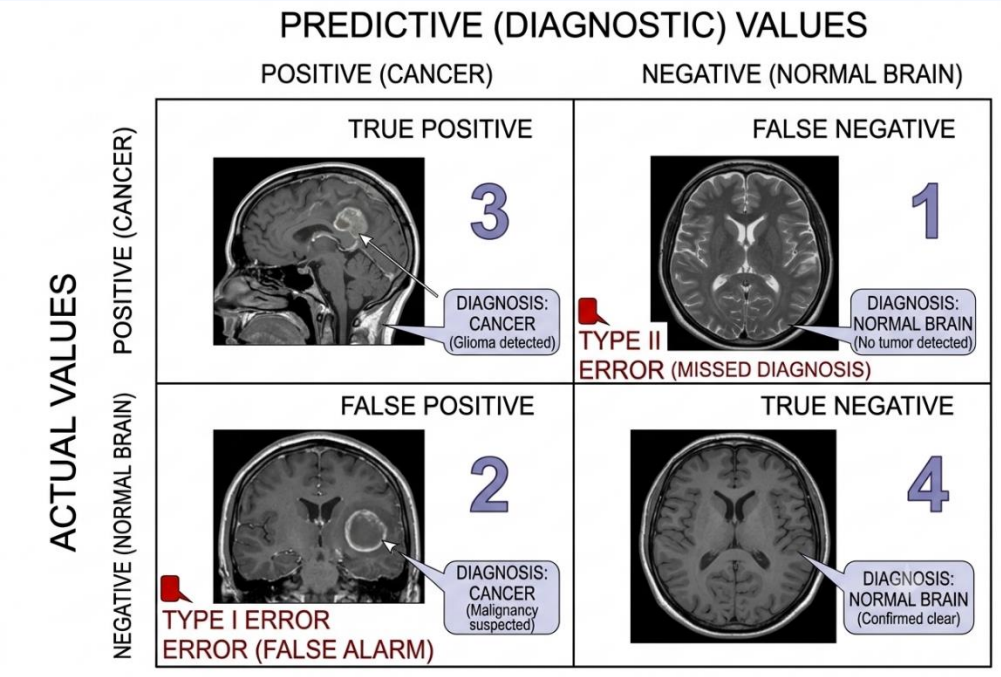
Decoding the confusion matrix

- ❑ A confusion matrix or error matrix is used for summarizing the performance of a classification algorithm.
- ❑ A confusion matrix is a performance evaluation tool used in machine learning that **summarizes the performance of a classification model** by tabulating true positive, true negative, false positive, and false negative predictions. It helps assess the accuracy and effectiveness of the model's predictions. The matrix is based on the concepts of **true positives (TP)**, **true negatives (TN)**, **false positives (FP)**, and **false negatives (FN)**.



Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

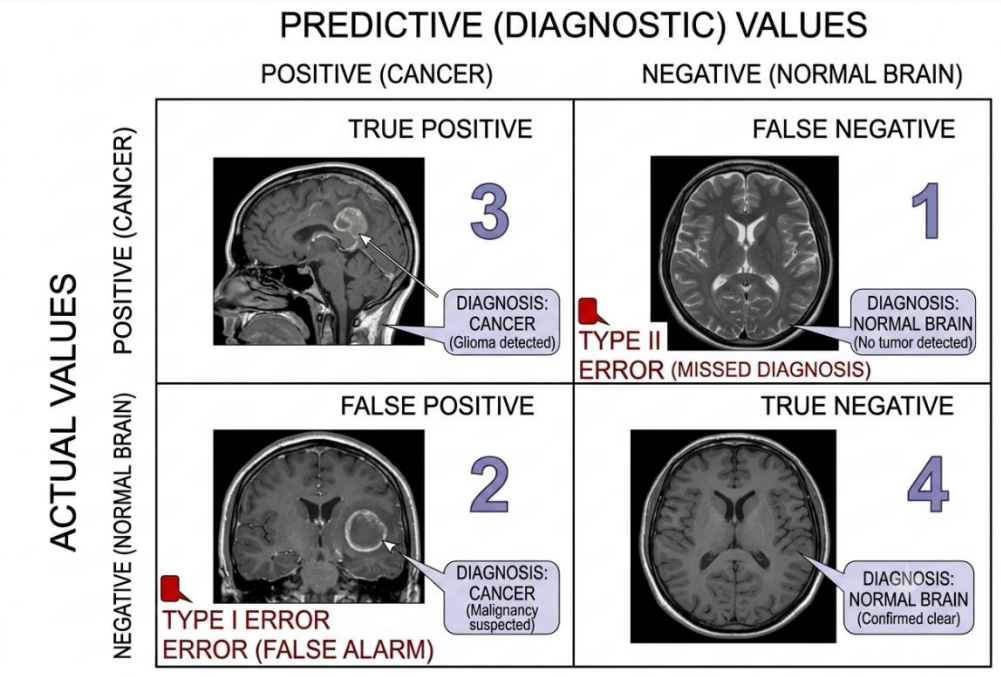


	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

	Predicated Values	
	1	0
1	y_true=1 y_pred=1	y_true=1 y_pred=0
0	y_true=0 y_pred=1	y_true=0 y_pred=0

Notes

(First term: If the model answers the prediction question correctly, we said True, else False)
(Second term: describes the predicted value)



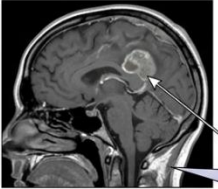
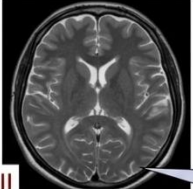
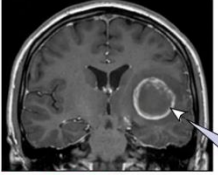
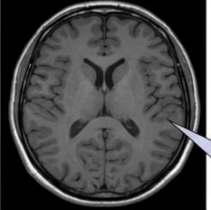
	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Actual Values

	Predicated Values	
	1	0
1	y_true=1 y_pred=1	y_true=1 y_pred=0
0	y_true=0 y_pred=1	y_true=0 y_pred=0

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

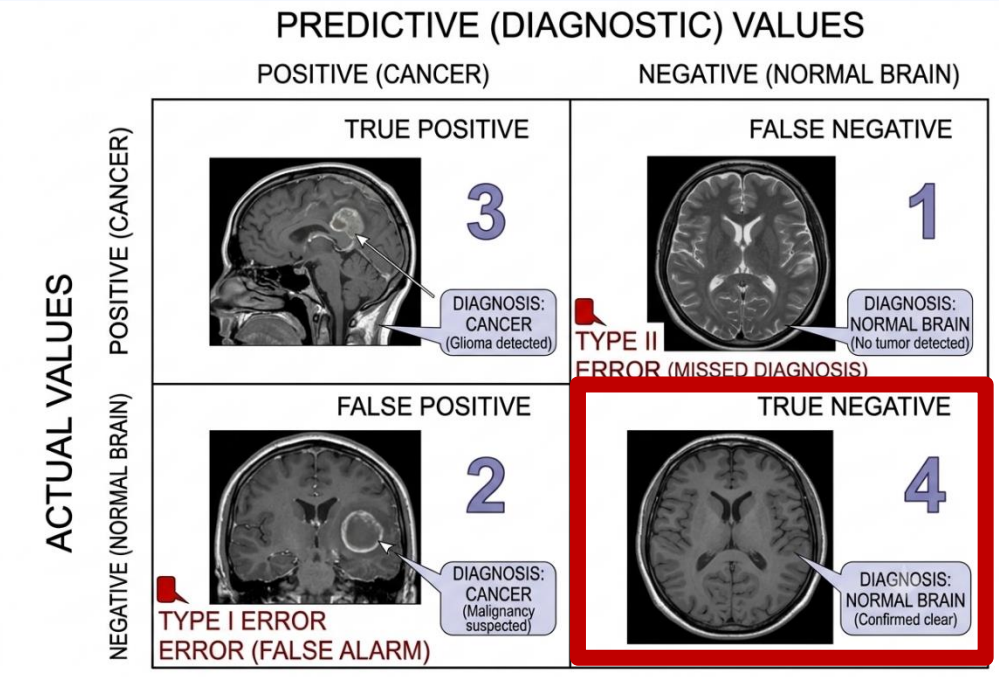
		PREDICTIVE (DIAGNOSTIC) VALUES	
		POSITIVE (CANCER)	NEGATIVE (NORMAL BRAIN)
ACTUAL VALUES	POSITIVE (CANCER)	TRUE POSITIVE  3 DIAGNOSIS: CANCER (Glioma detected)	FALSE NEGATIVE  1 DIAGNOSIS: NORMAL BRAIN (No tumor detected) TYPE II ERROR (MISSED DIAGNOSIS)
	NEGATIVE (NORMAL BRAIN)	FALSE POSITIVE  2 DIAGNOSIS: CANCER (Malignancy suspected) TYPE I ERROR ERROR (FALSE ALARM)	TRUE NEGATIVE  4 DIAGNOSIS: NORMAL BRAIN (Confirmed clear)



Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

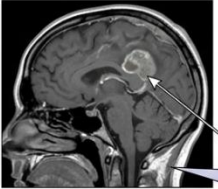
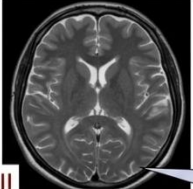
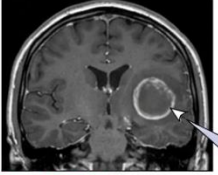
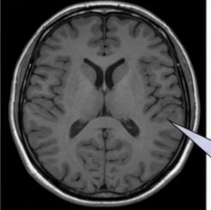


TN

Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

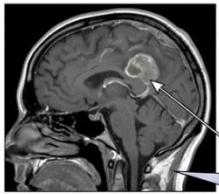
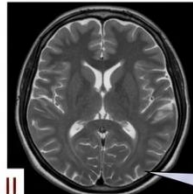
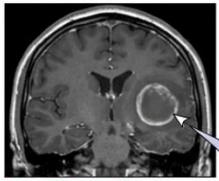
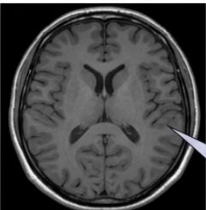
		PREDICTIVE (DIAGNOSTIC) VALUES	
		POSITIVE (CANCER)	NEGATIVE (NORMAL BRAIN)
ACTUAL VALUES	POSITIVE (CANCER)	TRUE POSITIVE  3 DIAGNOSIS: CANCER (Glioma detected)	FALSE NEGATIVE  1 DIAGNOSIS: NORMAL BRAIN (No tumor detected) TYPE II ERROR (MISSED DIAGNOSIS)
	NEGATIVE (NORMAL BRAIN)	FALSE POSITIVE  2 DIAGNOSIS: CANCER (Malignancy suspected) TYPE I ERROR (FALSE ALARM)	TRUE NEGATIVE  4 DIAGNOSIS: NORMAL BRAIN (Confirmed clear)



Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

		PREDICTIVE (DIAGNOSTIC) VALUES	
		POSITIVE (CANCER)	NEGATIVE (NORMAL BRAIN)
ACTUAL VALUES	POSITIVE (CANCER)	TRUE POSITIVE  3 DIAGNOSIS: CANCER (Glioma detected)	FALSE NEGATIVE  1 DIAGNOSIS: NORMAL BRAIN (No tumor detected) TYPE II
	NEGATIVE (NORMAL BRAIN)	FALSE POSITIVE  2 DIAGNOSIS: CANCER (Malignancy suspected) TYPE I ERROR ERROR (FALSE ALARM)	TRUE NEGATIVE  4 DIAGNOSIS: NORMAL BRAIN (Confirmed clear)

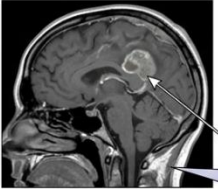
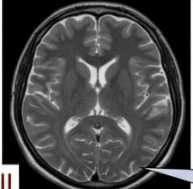
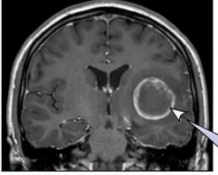
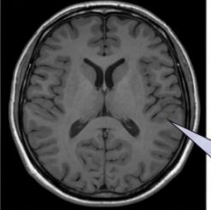


FN

Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

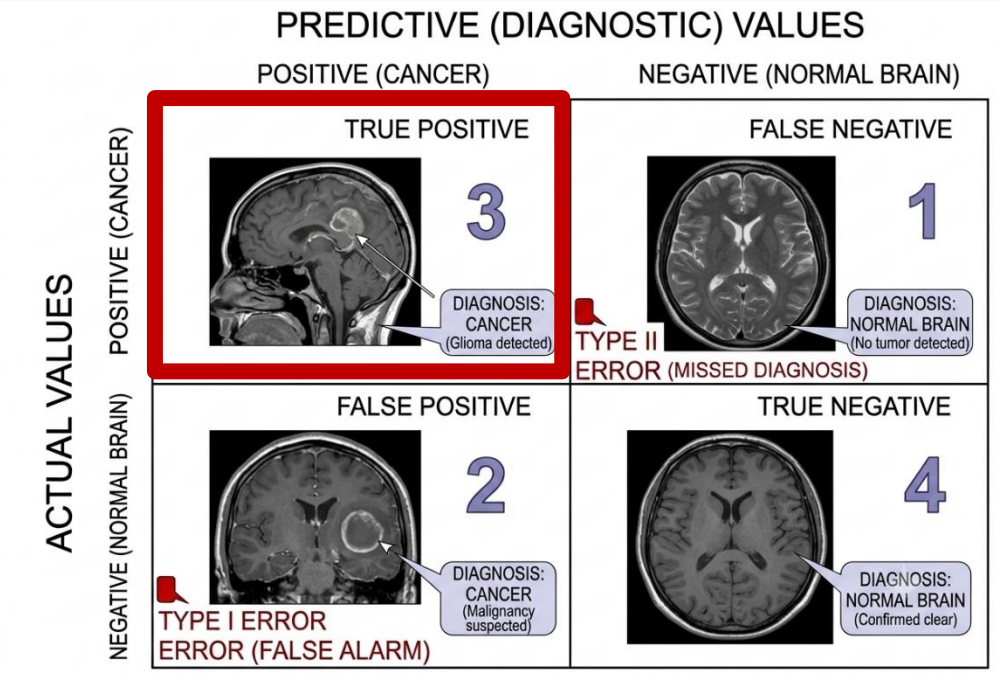
		PREDICTIVE (DIAGNOSTIC) VALUES	
		POSITIVE (CANCER)	NEGATIVE (NORMAL BRAIN)
ACTUAL VALUES	POSITIVE (CANCER)	TRUE POSITIVE  3 DIAGNOSIS: CANCER (Glioma detected)	FALSE NEGATIVE  1 DIAGNOSIS: NORMAL BRAIN (No tumor detected) TYPE II ERROR (MISSED DIAGNOSIS)
	NEGATIVE (NORMAL BRAIN)	FALSE POSITIVE  2 DIAGNOSIS: CANCER (Malignancy suspected) TYPE I ERROR ERROR (FALSE ALARM)	TRUE NEGATIVE  4 DIAGNOSIS: NORMAL BRAIN (Confirmed clear)



Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

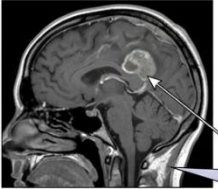
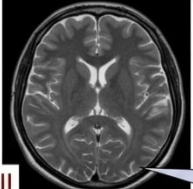
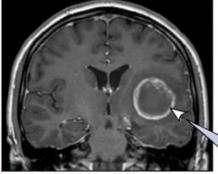
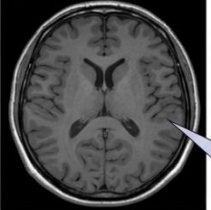


TP

Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)

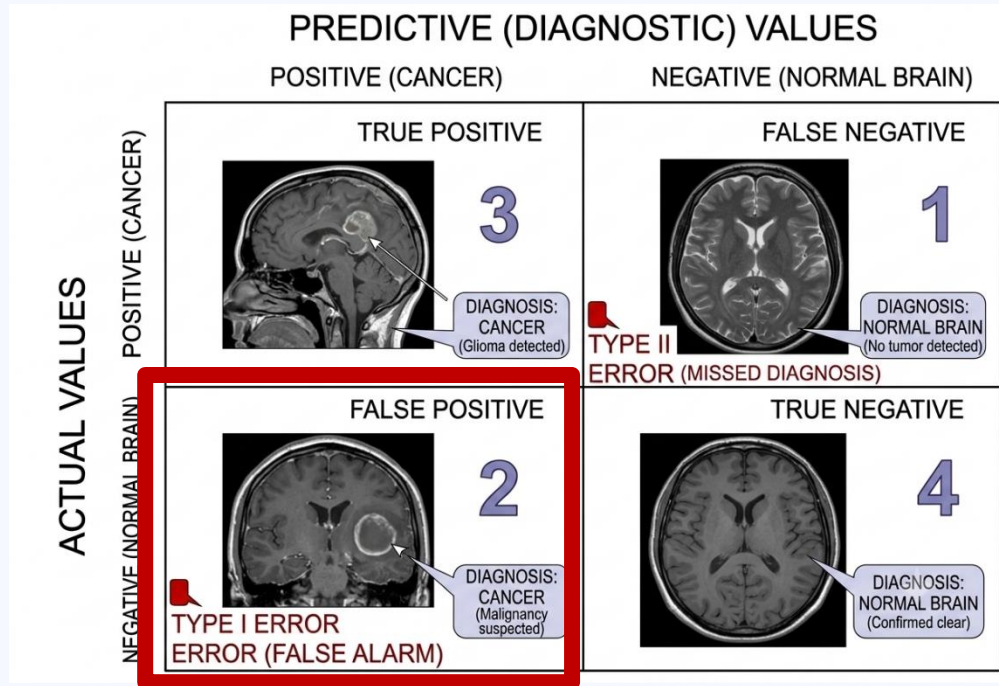
		PREDICTIVE (DIAGNOSTIC) VALUES	
		POSITIVE (CANCER)	NEGATIVE (NORMAL BRAIN)
ACTUAL VALUES	POSITIVE (CANCER)	TRUE POSITIVE  3 DIAGNOSIS: CANCER (Glioma detected)	FALSE NEGATIVE  1 DIAGNOSIS: NORMAL BRAIN (No tumor detected) TYPE II ERROR (MISSED DIAGNOSIS)
	NEGATIVE (NORMAL BRAIN)	FALSE POSITIVE  2 DIAGNOSIS: CANCER (Malignancy suspected) TYPE I ERROR ERROR (FALSE ALARM)	TRUE NEGATIVE  4 DIAGNOSIS: NORMAL BRAIN (Confirmed clear)



Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Notes

(Remember, we describe predicted values as Positive/Negative and actual values as True/False.)



FP

Actual values = ['N', 'C', 'N', 'C', 'N', 'N', 'C', 'N', 'C', 'N']
Predicted values = ['N', 'N', 'N', 'C', 'N', 'N', 'C', 'C', 'C', 'C']

Accuracy & Recall

❑ **Accuracy** is the proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

❑ **The true positive rate (TPR)**, or the proportion of all actual positives that were classified correctly as positives, is also known as **recall** (aka. **Sensitivity**).

Recall is mathematically defined as:

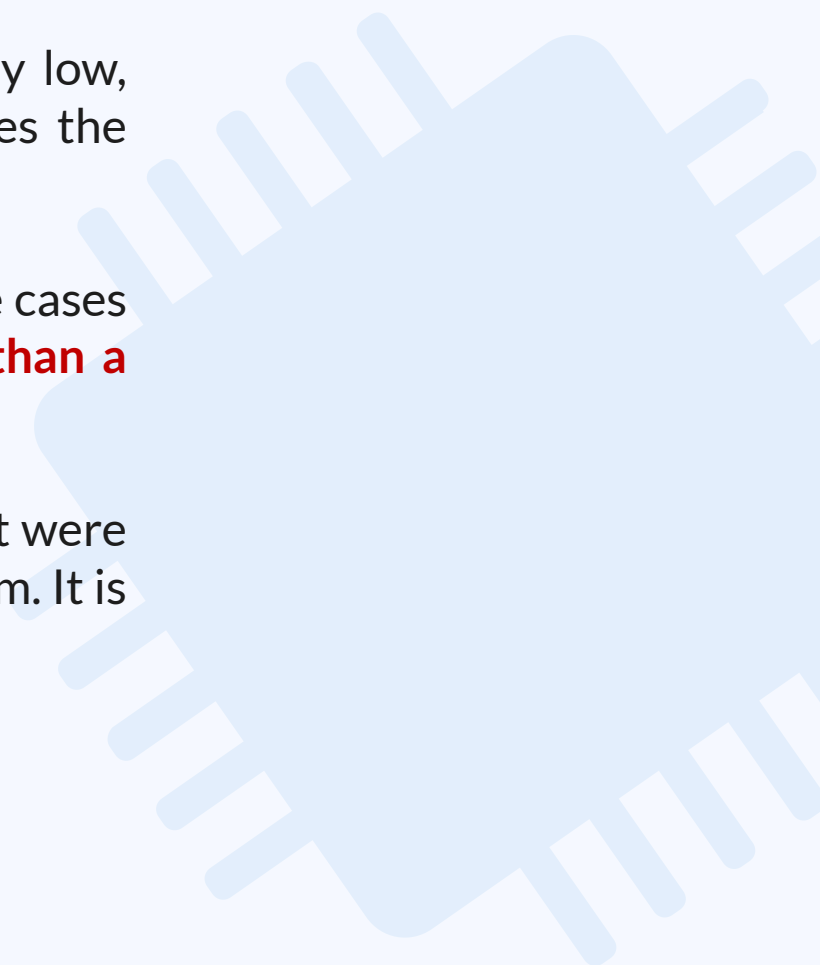
$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$



FPR

- ❑ In an **imbalanced dataset** where the number of actual positives is very low, **recall is a more meaningful metric than accuracy** because it measures the ability of the model to correctly identify all positive instances.
- ❑ For applications like disease prediction, correctly identifying the positive cases is crucial. **A false negative typically has more serious consequences than a false positive.**
- ❑ **The false positive rate (FPR)** is the proportion of all actual negatives that were classified incorrectly as positives, also known as the probability of false alarm. It is mathematically defined as:

$$\text{FPR} = \frac{\text{incorrectly classified actual negatives}}{\text{all actual negatives}} = \frac{FP}{FP + TN}$$



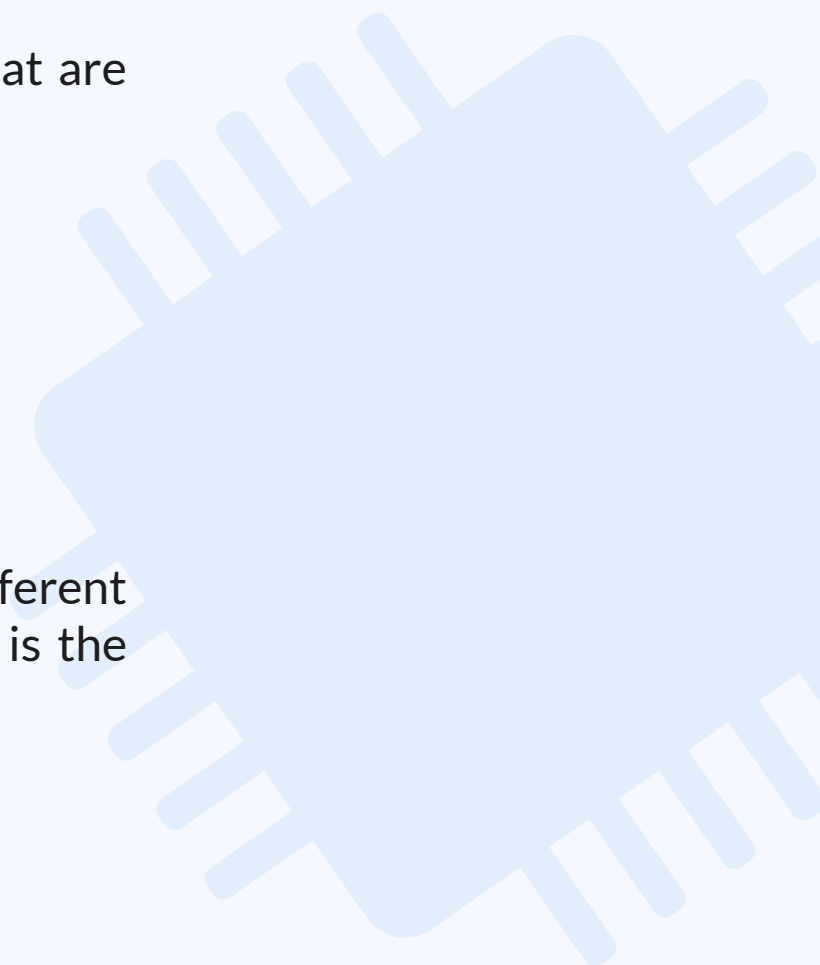
FPR

- ❑ **Precision** is the proportion of all the model's positive classifications that are actually positive. It is mathematically defined as:

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

- ❑ **F-score or F1-score**: It is difficult to compare two models with different Precision and Recall. So to make them comparable, we use F-Score. It is the Harmonic Mean of Precision and Recall.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

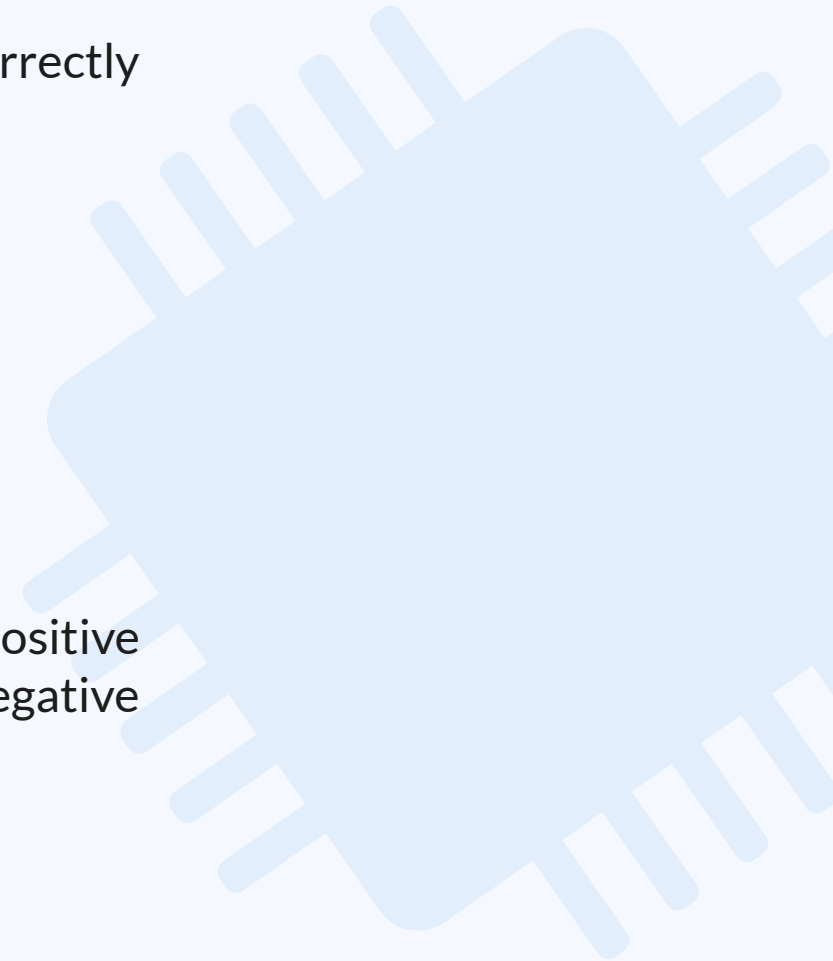


Specificity

- ❑ **Specificity** determines the proportion of actual negatives that are correctly identified.

$$\text{Specificity} = \frac{\text{True negative}}{\text{True negative} + \text{false positive}}$$

- ❑ **Sensitivity** measures the model's ability to correctly identify actual positive cases whereas specificity focuses on correctly identifying actual negative cases.



Case Study: Diabetes Dataset

```
from sklearn.metrics import confusion_matrix, roc_auc_score
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
```

In binary classification, executing `confusion_matrix(y_test, y_pred).ravel()` flattens a 2x2 confusion matrix into a 1D array, unpacking its values in the exact order of **True Negatives (TN)**, **False Positives (FP)**, **False Negatives (FN)**, and **True Positives (TP)**

Case Study: Diabetes Dataset

Dataset Overview

- **Observations:** 768 patient records (500 negative, 258 positive)

```
sensitivity = tp / (tp + fn) # Recall: How many actual diabetes cases were detected?  
specificity = tn / (tn + fp) # How many healthy patients were correctly cleared?  
auc_roc = roc_auc_score(y_test, y_prob)
```

```
print(f"--- CLINICAL METRICS ---")  
print(f"Sensitivity (Recall) : {sensitivity:.2%}")  
print(f"Specificity          : {specificity:.2%}")  
print(f"AUC-ROC Score          : {auc_roc:.3f}\n")
```

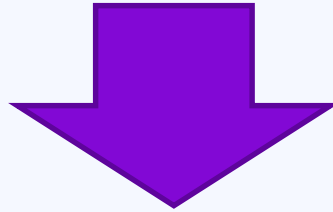
```
--- CLINICAL METRICS ---  
Sensitivity (Recall) : 50.00%  
Specificity          : 82.00%  
AUC-ROC Score        : 0.813
```

- ❑ **Sensitivity (Recall):** The ability of a model to correctly identify positive cases (True Positive Rate).
- ❑ **Specificity:** The ability of a model to correctly exclude negative cases (True Negative Rate).
- ❑ **AUC-ROC Score:** The overall performance measurement across all classification thresholds, where 1.0 represents a perfect model and 0.5 represents random guessing.

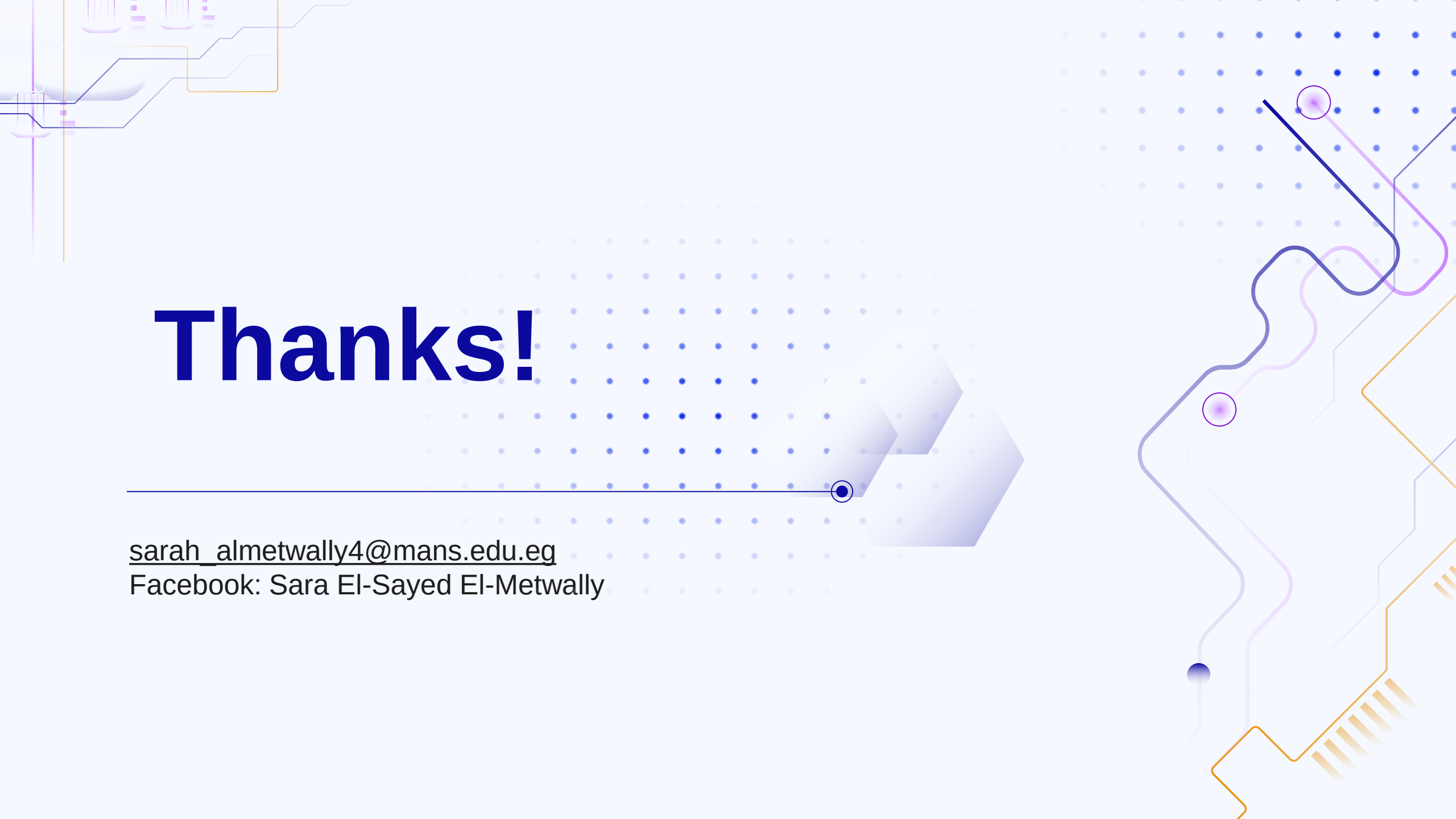
Assignment: Train Logistic Regression on the Heart Disease Dataset

url = "<https://raw.githubusercontent.com/sharmaroshan/Heart-UCI-Dataset/master/heart.csv>"

Feature	Meaning
age	Age in years
sex	1 = male, 0 = female
cp	Chest pain type (0–3)
trestbps	Resting blood pressure (mm Hg)
chol	Serum cholesterol (mg/dl)
fbs	Fasting blood sugar > 120 mg/dl (1 = true)
restecg	Resting ECG results (0–2)
thalach	Max heart rate achieved
exang	Exercise-induced angina (1 = yes)
oldpeak	ST depression induced by exercise
slope	Slope of peak exercise ST segment
ca	Number of major vessels colored by fluoroscopy (0–4)
thal	Thalassemia result (0–3)
target	Heart disease present (1) or not (0)



<https://forms.gle/xuiLRFWfdxkrChHE9>



Thanks!

sarah_almetwally4@mans.edu.eg

Facebook: Sara El-Sayed El-Metwally